OKay so im building an opengl coursework project, ive been messing aorund with a few different techniques and want to change up my idea fromm a space/flying around thing to a static ish scene of a lightsaber hilt, where youre able to rotate the hilt then turn on/off the lightsaber blade, i want you to help me cut down my code to being more in line with what i want, my code is a bit of a mess so right now my focus is tidying the code and being careful in how i remove parts to ensure it continues to work at each stage, before you start to assist i will show you my main files i intend to edit then we will discuss how to change things for my coursework.

here is my scenebasic.cpp:

```
#include "scenebasic_uniform.h"

#include <cstdio>
#include <cstdlib>

#include <string>
using std::string;

#include <sstream>
#include <iostream>
using std::cerr;
using std::endl;

#include <GLFW/glfw3.h>
#include "helper/glutils.h"
#include "helper/texture.h"

using glm::vec3;
using glm::vec4;
using glm::mat3;
using glm::mat4;

SceneBasic_Uniform::SceneBasic_Uniform() : tPrev(0), angle(0.0f),
rotSpeed(glm::pi<float>() / 2.0f), plane(100.0f, 100.0f, 100, 100),
teapot(14, glm::mat4(1.0f)), torus(1.75f * 0.75f, 0.75f * 0.75f, 50, 50),
sky(100.0f) {
    R2Mesh = ObjMesh::load("media/Low_Poly_R2D2.obj", true);
    C1Mesh = ObjMesh::load("media/C1-10P_obj.obj", true);
    XWingMesh = ObjMesh::load("media/X-Wing.obj", true);
    LightsaberMesh = ObjMesh::load("media/Lightsaber_01.obj", true);
}

void SceneBasic_Uniform::initScene()
{
    compile();
    prog.use();
    glEnable(GL_DEPTH_TEST);

    model = mat4(1.0f);
    view = glm::lookAt(vec3(5.0f, 7.5f, 7.5f), vec3(0.0f, 0.75f, 0.0f),
vec3(0.0f, 1.0f, 0.0f));
    //view = glm::lookAt(camPos, camPos + camFront, camUp);

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        // Start with cursor captured (FPS-style)
        glfwSetInputMode(win, GLFW_CURSOR, cursorCaptured ?
GLFW_CURSOR_DISABLED : GLFW_CURSOR_NORMAL);

        // Seed mouse position to avoid a huge first delta
        glfwGetCursorPos(win, &lastMouseX, &lastMouseY);
        firstMouse = true;
    }

    projection = mat4(1.0f);
    rotateModel = mat4(1.0f);
    rotateModel = glm::translate(rotateModel, vec3(4.0f, 0.26f, 5.0f));
    angle = 0.0f;
    float x, z;
    /*for (int i = 0; i < 3; i++) {
        std::stringstream name;
        name << "Lights[" << i << "].Position";
        x = 2.0f * cosf((glm::two_pi<float>() / 3) * i);
        z = 2.0f * sinf((glm::two_pi<float>() / 3) * i);
        prog.setUniform(name.str().c_str(), view * glm::vec4(x, 1.2f, z + 1.0f,
1.0f));

    }*/

    //prog.setUniform("lights[0].L", vec3(0.0f, 0.0f, 0.8f) / 2.0f);
    //prog.setUniform("lights[1].L", vec3(0.0f, 0.8f, 0.0f) / 2.0f);
```

```
    //prog.setUniform("lights[2].L", vec3(0.8f, 0.0f, 0.0f) / 2.0f);

    //prog.setUniform("lights[0].La", vec3(0.0f, 0.0f, 0.2f));
    //prog.setUniform("lights[1].La", vec3(0.0f, 0.2f, 0.0f));
    //prog.setUniform("lights[2].La", vec3(0.2f, 0.0f, 0.0f));

    //blinnPhongSpot stuff
    //prog.setUniform("Spot.L", vec3(0.7f));
    //prog.setUniform("Spot.La", vec3(0.5f));
    //prog.setUniform("Spot.Exponent", 25.0f);
    //prog.setUniform("Spot.Cutoff", glm::radians(100.0f));

    prog.setUniform("Light.L", vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    cubeTex =
Texture::loadCubeMap("media/texture/cube/space/space");
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);

    C1diffuseTexture =
Texture::loadTexture("media/texture/Chopper_BaseColor.png");
    C1normalMap =
Texture::loadTexture("media/texture/Chopper_Normal.png");
    R2diffuseTexture =
Texture::loadTexture("media/texture/R2_diffuse_green.png");
    R2normalMap =
Texture::loadTexture("media/texture/R2_normal.png");
    LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_01_lambert1_BaseColor.
png");
    LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_01_lambert1_Normal.pn
g");
    XWingDiffuseTexture =
Texture::loadTexture("media/texture/xwing_main.png");
    XWingNormalMap =
Texture::loadTexture("media/texture/xwing_main_n.png");
}

void SceneBasic_Uniform::compile()
{
    try {
        prog.compileShader("shader/basic_uniform.vert");
        prog.compileShader("shader/basic_uniform.frag");
        prog.link();
        prog.use();

        skyboxShader.compileShader("shader/skybox.vert");
        skyboxShader.compileShader("shader/skybox.frag");
        skyboxShader.link();
        skyboxShader.use();
    }
    catch (GLSLProgramException& e) {
        cerr << e.what() << endl;
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update(float t)
{
    float deltaT = t - tPrev;
    if (tPrev == 0.0f)
        deltaT = 0.0f;
    tPrev = t;
    angle += 0.25f * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();

    /*if (this->m_animate) {
        angle += rotSpeed * deltaT;
        if (angle > glm::two_pi<float>())
            angle -= glm::two_pi<float>();
    }*/
    updateCamera(deltaT);
    render();

}

void SceneBasic_Uniform::render()
{

    prog.use();

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    //blinnPhongSpot stuff
```

```
    //vec4 lightPos = vec4(10.0f * cos(angle), 10.0f, 10.0f * sin(angle),
1.0f);
    //prog.setUniform("Spot.Position", vec3(view * lightPos));
    //glm::mat3 normalMatrix = glm::mat3(vec3(view[0]), vec3(view[1]),
vec3(view[2]));
    //prog.setUniform("Spot.Direction", normalMatrix * vec3(-lightPos));

    //moving light
    //vec4 lightPos = vec4(10.0f * cos(angle), 10.0f, 10.0f * sin(angle),
1.0f);
    //prog.setUniform("Light.Position", view * lightPos);

    //glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f)); // tweak this
    //glm::vec3 lightPosWorld = -lightDirWorld * 200.0f; // far away in
opposite direction
    //prog.setUniform("Light.Position", view * glm::vec4(lightPosWorld,
0.0f));

    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));

    //prog.setUniform("Material.Kd", 1.0f, 0.4f, 0.7f);
    prog.setUniform("Material.Ks", vec3(0.5f));
    prog.setUniform("Material.Ka", vec3(0.5f));
    prog.setUniform("Material.Shininess", 180.0f);
    rotateModel = glm::translate(rotateModel, vec3(-0.9f, 0.0f, -0.9f));

    model = mat4(1.0f);
    model = glm::translate(model, vec3(1.0f, 2.0f, -4.0f));
    model = glm::rotate(model, glm::radians(70.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(3.0f));
    setMatrices();
    cube.render();

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, XWingDiffuseTexture);

    //glActiveTexture(GL_TEXTURE0);
    //glBindTexture(GL_TEXTURE_2D, etaDiffuse2Texture);

    //glActiveTexture(GL_TEXTURE2);
    //glBindTexture(GL_TEXTURE_2D, etaDiffuse3Texture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, XWingNormalMap);

    //prog.setUniform("Material.Kd", 0.0f, 0.5f, 1.0f);
    prog.setUniform("Material.Ks", vec3(0.5f));
    prog.setUniform("Material.Ka", vec3(0.8f));
    prog.setUniform("Material.Shininess", 180.0f);

    /*model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, 4.0f, 0.0f));
    model = glm::rotate(model, glm::radians(45.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(1.0f));
    setMatrices();
    XWingMesh->render();*/

    // --- X-Wing follows camera ---
    glm::vec3 forward = glm::normalize(camFront);
    glm::vec3 right = glm::normalize(glm::cross(forward, camUp));
    glm::vec3 up = glm::normalize(glm::cross(right, forward));

    // Offsets relative to camera
    float followForward = 8.2f;   // in front of camera
    float followDown = -2.7f;  // below camera (negative = down)
    float followRight = 0.0f;   // sideways

    glm::vec3 xwingPos = camPos + forward * followForward + up *
followDown + right * followRight;

    // Build an orientation so the ship points the same way as the
camera.
    // NOTE: depending on how the OBJ is authored, you may need a
fixed extra rotation after this.
    glm::mat4 shipRot = glm::mat4(1.0f);
    shipRot[0] = glm::vec4(right, 0.0f);
    shipRot[1] = glm::vec4(up, 0.0f);
    shipRot[2] = glm::vec4(-forward, 0.0f); // -forward is common for
OpenGL forward axis

    model = glm::mat4(1.0f);
    model = glm::translate(model, xwingPos);
    model *= shipRot;
```

```
    model = model * glm::rotate(glm::mat4(1.0f), glm::radians(shipRoll),
glm::vec3(0, 0, 1));

    model = glm::scale(model, glm::vec3(0.5f));
    setMatrices();
    XWingMesh->render();


    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, C1diffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, C1normalMap);

    //prog.setUniform("Material.Kd", 1.0f, 0.5f, 0.0f);
    prog.setUniform("Material.Ks", vec3(0.5f));
    prog.setUniform("Material.Ka", vec3(0.8f));
    prog.setUniform("Material.Shininess", 80.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, 2.0f, 5.0f));
    model = glm::rotate(model, glm::radians(45.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(2.0f));
    setMatrices();
    C1Mesh->render();

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);

    //prog.setUniform("Material.Kd", 1.0f, 1.0f, 0.0f);
    prog.setUniform("Material.Ks", vec3(0.5f));
    prog.setUniform("Material.Ka", vec3(0.8f));
    prog.setUniform("Material.Shininess", 80.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(-5.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));

    setMatrices();
    LightsaberMesh->render();


    //prog.setUniform("Material.Kd", 0.76f, 0.60f, 0.42f);
    prog.setUniform("Material.Ks", vec3(0.2f));
    prog.setUniform("Material.Ka", vec3(0.2f));
    prog.setUniform("Material.Shininess", 18.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(5.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(90.0f), vec3(1.0f, 0.0f, 0.0f));

    setMatrices();
    torus.render();


    //prog.setUniform("Material.Kd", 0.1f, 0.8f, 0.1f);
    prog.setUniform("Material.Ks", vec3(0.0f));
    prog.setUniform("Material.Ka", vec3(0.1f));
    prog.setUniform("Material.Shininess", 180.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, -0.45f, 0.0f));
    setMatrices();
    //plane.render();

    skyboxShader.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
    skyboxShader.setUniform("SkyboxTex", 0);

    glm::mat4 skyView = glm::mat4(glm::mat3(view));
    glm::mat4 skyMVP = projection * skyView;
    skyboxShader.setUniform("MVP", skyMVP);
    sky.render();
}

void SceneBasic_Uniform::setMatrices() {
    mat4 mv = view * model;
    prog.setUniform("ModelViewMatrix", mv);
    prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]),
vec3(mv[2])));
    prog.setUniform("MVP", projection * mv);
}
```

```
void SceneBasic_Uniform::resize(int w, int h)
{
    glViewport(0, 0, w, h);
    width = w;
    height = h;
    projection = glm::perspective(glm::radians(fovCurrent), (float)w / h,
0.3f, 100.0f);
}

void SceneBasic_Uniform::updateCamera(float dt)
{
    GLFWwindow* win = glfwGetCurrentContext();
    if (!win) return;

    // --- ESC behaviour (edge-triggered) ---
    bool escDown = (glfwGetKey(win, GLFW_KEY_ESCAPE) ==
GLFW_PRESS);
    if (escDown && !escWasDown)
    {
        if (cursorCaptured)
        {
            // Release mouse
            cursorCaptured = false;
            glfwSetInputMode(win, GLFW_CURSOR,
GLFW_CURSOR_NORMAL);
            firstMouse = true;
        }
        else
        {
            // Cursor already free -> close program
            glfwSetWindowShouldClose(win, GLFW_TRUE);
        }
    }
    escWasDown = escDown;

    // --- Mouse look (only while captured) ---
    if (cursorCaptured)
    {
        double mx, my;
        glfwGetCursorPos(win, &mx, &my);

        if (firstMouse)
        {
            lastMouseX = mx;
            lastMouseY = my;
            firstMouse = false;
        }

        double dx = mx - lastMouseX;
        double dy = lastMouseY - my; // up = positive pitch

        lastMouseX = mx;
        lastMouseY = my;

        yaw += static_cast<float>(dx) * mouseSensitivity;
        pitch += static_cast<float>(dy) * mouseSensitivity;

        // Clamp pitch to avoid flip
        if (pitch > 89.0f)  pitch = 89.0f;
        if (pitch < -89.0f) pitch = -89.0f;

        // Rebuild camFront from yaw/pitch
        glm::vec3 front;
        front.x = cos(glm::radians(yaw)) * cos(glm::radians(pitch));
        front.y = sin(glm::radians(pitch));
        front.z = sin(glm::radians(yaw)) * cos(glm::radians(pitch));
        camFront = glm::normalize(front);
    }

    // --- Movement speed ---
    float speed = camSpeed * dt;
    if (glfwGetKey(win, GLFW_KEY_LEFT_SHIFT) == GLFW_PRESS)
        speed *= 2.0f;

    bool boosting = (glfwGetKey(win, GLFW_KEY_LEFT_SHIFT) ==
GLFW_PRESS);

    float targetFov = boosting ? boostFov : baseFov;

    // Smooth towards target FOV (frame-rate independent-ish)
    float k = 1.0f - expf(-fovSmoothSpeed * dt);
    fovCurrent = fovCurrent + (targetFov - fovCurrent) * k;

    // Rebuild projection with updated FOV
    projection = glm::perspective(glm::radians(fovCurrent), (float)width /
(float)height, 0.3f, 100.0f);
```

```
    // --- Free-fly directions (true 6DOF translation) ---
    glm::vec3 forward = camFront;
    if (glm::length(forward) > 0.0001f) forward = glm::normalize(forward);

    glm::vec3 right = glm::cross(forward, camUp);
    if (glm::length(right) > 0.0001f) right = glm::normalize(right);

    // (Optional) re-orthonormalise up so drift doesn't build up over time
    glm::vec3 up = glm::cross(right, forward);
    if (glm::length(up) > 0.0001f) up = glm::normalize(up);
    float strafe = 0.0f;

    // --- WASD translate in camera space ---
    if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS) camPos +=
forward * speed;
    if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS) camPos -=
forward * speed;
    if (glfwGetKey(win, GLFW_KEY_A) == GLFW_PRESS) {
        camPos -= right * speed;
        strafe -= 1.0f;
    }
    if (glfwGetKey(win, GLFW_KEY_D) == GLFW_PRESS) {
        camPos += right * speed;
        strafe += 1.0f;
    }

    // Bank opposite sign depending on what feels right. Start with this:
    shipRollTarget = -strafe * maxShipRoll;

    // Smooth roll
    float kroll = 1.0f - expf(-shipRollResponsiveness * dt);
    shipRoll = shipRoll + (shipRollTarget - shipRoll) * kroll;

    // Vertical strafe (camera up)
    if (glfwGetKey(win, GLFW_KEY_SPACE) == GLFW_PRESS)        camPos
+= up * speed;
    if (glfwGetKey(win, GLFW_KEY_LEFT_CONTROL) == GLFW_PRESS)
camPos -= up * speed;

    // Apply view matrix (use the re-orthonormalised up)
    view = glm::lookAt(camPos, camPos + forward, up);
}

scenebasic_uniform.h:
#ifndef SCENEBASIC_UNIFORM_H
#define SCENEBASIC_UNIFORM_H

#include "helper/scene.h"

#include <glm/glm.hpp>
#include <glad/glad.h>
#include "helper/glslprogram.h"
#include "helper/torus.h"
#include "helper/teapot.h"
#include "helper/plane.h"
#include "helper/objmesh.h"
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include "helper/cube.h"
#include "helper/skybox.h"


class SceneBasic_Uniform : public Scene
{
private:
    glm::mat4 rotationMatrix;
    SkyBox sky;
    Torus torus;
    Teapot teapot;
    Plane plane;
    Cube cube;
    std::unique_ptr<ObjMesh> R2Mesh;
    std::unique_ptr<ObjMesh> C1Mesh;
    std::unique_ptr<ObjMesh> XWingMesh;
    std::unique_ptr<ObjMesh> LightsaberMesh;
    glm::mat4 rotateModel;
    GLuint C1diffuseTexture, C1normalMap;
    GLuint R2diffuseTexture, R2normalMap;
    GLuint LSdiffuseTexture, LSnormalMap;
    GLuint XWingDiffuseTexture, XWingNormalMap;
    GLuint cubeTex;


    float tPrev = 0.0f;
    float angle = 0.0f;
    float rotSpeed;
```

```
    GLSLProgram prog, skyboxShader;
    void compile();
    void setMatrices();

    // Camera stuff
    glm::vec3 camPos = glm::vec3(5.0f, 7.5f, 7.5f);
    glm::vec3 camFront = glm::normalize(glm::vec3(0.0f, 0.75f, 0.0f) -
camPos);
    glm::vec3 camUp = glm::vec3(0.0f, 1.0f, 0.0f);
    float baseFov = 70.0f;
    float boostFov = 82.0f;          // try 78–90
    float fovCurrent = 70.0f;
    float fovSmoothSpeed = 10.0f;    // higher = snappier

    float camSpeed = 6.0f; // units/sec
    void updateCamera(float dt);

    float yaw = -90.0f;
    float pitch = 0.0f;
    float mouseSensitivity = 0.12f;
    bool firstMouse = true;
    double lastMouseX = 0.0;
    double lastMouseY = 0.0;
    float shipRoll = 0.0f;          // current roll in degrees
    float shipRollTarget = 0.0f;    // target roll in degrees
    float maxShipRoll = 35.0f;      // try 20–45
    float shipRollResponsiveness = 12.0f; // higher = snappier

    bool cursorCaptured = true;
    bool escWasDown = false;

public:
    SceneBasic_Uniform();

    void initScene();
    void update( float t );
    void render();
    void resize(int, int);

};

#endif // SCENEBASIC_UNIFORM_H

basic_uniform.frag:
#version 460

in vec3 LightDir;
in vec3 ViewDir;
in vec2 TexCoord;

layout (location = 0) out vec4 FragColor;
layout (binding = 0) uniform sampler2D DiffuseTex;
layout (binding = 1) uniform sampler2D NormalMapTex;
layout (binding = 2) uniform sampler2D DiffuseTex2;

//uniform struct LightInfo{
//    vec4 Position;
//    vec3 La;
//    vec3 L;
//}lights[3];

uniform struct MaterialInfo{
    vec3 Kd;
    vec3 Ka;
    vec3 Ks;
    float Shininess;
}Material;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct FogInfo{
    float MaxDistance;
    float MinDistance;
    vec3 Colour;
```

```
}Fog;

const int levels = 5;
const float scaleFactor = 1.0/levels;



//can get rid of this section if brave enough
//vec3 phongModel(int light, vec3 position, vec3 n){
//    vec3 ambient = lights[light].La * Material.Ka;
//    vec3 s = normalize(vec3(lights[light].Position.xyz) - position);
//    float sDotN = max(dot(s, n), 0.0);
//    vec3 diffuse = Material.Kd * sDotN;
//    vec3 spec = vec3(0.0);
//    if (sDotN > 0.0){
//        vec3 v = normalize(-position.xyz);
//        vec3 r = reflect(-s, n);
//        spec = Material.Ks * pow(max(dot(r, v), 0.0), Material.Shininess);
//    }
//    return ambient + (diffuse + spec) * lights[light].L;
//}



vec3 blinnPhong(vec3 n) {
    vec3 diffuse = vec3(0.0f);
    vec3 spec = vec3(0.0f);

    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Light.La * Material.Ka * texColour;
    vec3 s = normalize(LightDir);
    //float sDotN = max(dot(s, n), 0.0);
    float sDotN = max(dot(s, n), 0.0);

    diffuse = texColour * sDotN;

    if (sDotN > 0.0) {
        vec3 v = normalize(ViewDir);
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }
    return ambient + (diffuse + spec) * Light.L;
}



//can get rid of this section if brave enough
vec3 blinnPhongSpot(vec3 position, vec3 n)
{
    vec3 ambient = Spot.La * Material.Ka;

    vec3 s = normalize(vec3(Spot.Position.xyz-position));

    vec3 diffuse = vec3(0.0f);

    vec3 spec = vec3(0.0f);

    float cosAng = dot(-s, normalize(Spot.Direction));
    float angle = acos(cosAng);
    float spotScale = 0.0;

    if (angle < Spot.Cutoff)
    {
        spotScale = pow(cosAng, Spot.Exponent);
        float sDotN = max(dot(n,s),0.0);
        diffuse = Material.Kd*floor(sDotN * levels) * scaleFactor;


        if(sDotN > 0.0)
        {
            vec3 v = normalize(-position.xyz);
            vec3 r = reflect(-s,n);
            spec = Material.Ks * pow(max(dot(r,v), 0.0), Material.Shininess);
        }
    }

    return ambient + spotScale *Spot.L*(diffuse + spec);
}



void main() {
//fog stuff
    //float distance = abs(Position.z);
    //float fogFactor = (Fog.MaxDistance - distance) / (Fog.MaxDistance
- Fog.MinDistance);
```

```
    //fogFactor = clamp(fogFactor, 0.0, 1.0);
    //vec3 shadeColour = blinnPhong(Position, normalize(Normal));
    //vec3 Colour = mix(Fog.Colour, shadeColour, fogFactor);  // --
remember to put Colour instead of shadeColour when i want fog back
//phong
    //vec3 Colour = vec3(0.0);
    //for (int i = 0; i <3; i++)
    //    Colour += phongModel(i, Position, Normal);
//blinnphongspot
    //vec3 shadeColour = blinnPhongSpot(Position, normalize(Normal));
    //vec3 Colour = shadeColour;
    //FragColor = vec4(Colour, 1.0);
//blinnphong
    vec3 normal = texture(NormalMapTex, TexCoord).xyz;
    //normal.xy = 2.0 * normal.xy - 1.0;
    normal = normal * 2.0 - 1.0;
    normal = normalize(normal);
    FragColor = vec4(blinnPhong(normalize(normal)), 1.0);
}

basic_uniform.vert:
#version 460

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec3 VertexNormal;
layout (location = 2) in vec2 VertexTexCoord;
layout (location = 3) in vec4 VertexTangent;

out vec3 LightDir;
out vec3 ViewDir;
out vec2 TexCoord;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 ProjectionMatrix;
uniform mat4 MVP;

out vec3 LightIntensity, n, pos;

void main()
{
    //vec3 normal = normalize(NormalMatrix * VertexNormal);
    //vec3 tangent = normalize(NormalMatrix * vec3(VertexTangent));
    //vec3 binormal = normalize(cross(normal, tangent)) *
VertexTangent.w;
    //vec3 Position = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;

    //mat3 toObjectLocal = mat3(
    // tangent.x, binormal.x, normal.x,
    // tangent.y, binormal.y, normal.y,
    // tangent.z, binormal.z, normal.z
    //);


    //LightDir = toObjectLocal * (Light.Position.xyz - Position);
    //ViewDir = toObjectLocal * normalize(-Position);
    //TexCoord = VertexTexCoord;

    //gl_Position = MVP * vec4(VertexPosition, 1.0);
    vec3 N = normalize(NormalMatrix * VertexNormal);
    vec3 T = normalize(NormalMatrix * vec3(VertexTangent.xyz));

    // Make T orthogonal to N (helps a LOT with noisy normal maps)
    T = normalize(T - N * dot(N, T));

    vec3 B = cross(N, T) * VertexTangent.w;
```

ChatGPT 5.2 Thinking

Share

```
    mat3 TBN = mat3(T, B, N);

    mat3 invTBN = transpose(TBN);

    vec3 Position = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;

    vec3 lightDir;
    if (Light.Position.w == 0.0)
        lightDir = normalize(mat3(ModelViewMatrix) * Light.Position.xyz);
// directional
    else
        lightDir = Light.Position.xyz - Position;       // point light
    LightDir = invTBN * lightDir;
```

```
    //LightDir = TBN * (Light.Position.xyz - Position);

    ViewDir  = invTBN * normalize(-Position);
    TexCoord = VertexTexCoord;

    gl_Position = MVP * vec4(VertexPosition, 1.0);
}
```

Thought for 19s ›

Nice — we can absolutely get this project "back under control" in small, safe cuts.

Below is a **careful refactor plan** that keeps things compiling at each step, plus the **first concrete cut** I'd do based on the files you pasted (remove the space-flight / extra models, keep a single lightsaber hilt, keep lighting + normal map).

What you want (target shape)

Scene: one lightsaber hilt on a plinth/ground (optional)
Controls:

- Rotate the **hilt** (not free-fly the camera)
- Toggle the **blade** on/off

Rendering:

- Keep your current **Blinn-Phong + normal map** shader for the hilt
- Render the blade as a simple **emissive/glowy** object (separate shader or a "special case")

Refactor strategy (safe checkpoints)

Do this with commits (or zip backups) so you can revert easily.

Checkpoint A — "Only lightsaber hilt renders"

Goal: remove everything except:

- `LightsaberMesh`
- `LSdiffuseTexture`, `LSnormalMap`
- one light setup
- your shader `basic_uniform.*`

Remove/disable:

- XWing follow-camera block
- C1/R2 models + textures
- torus/teapot/cube (unless you want a simple stand)
- skybox (optional; I'd remove it early)

✅ This should still compile + show a hilt.

Checkpoint B — "Rotate hilt with input (no WASD flight)"

Goal: camera becomes mostly fixed; hilt rotates.

Simplest approach:

- Keep a fixed `view = lookAt(...)`
- Add `float hiltYaw`, `hiltPitch` (or just yaw)
- Use mouse drag (or arrow keys) to change those
- Apply rotation to the hilt's `model` matrix

✅ Still your same shader.

Checkpoint C — "Blade toggle"

Goal: press a key to toggle blade visibility.

Best low-risk approach: render blade using a **second tiny shader** (unlit/emissive colour) and a simple primitive you already have (`Cube`) scaled into a long thin "blade".

✅ Doesn't touch your normal-mapped shader at all (so less chance of breaking it).

Checkpoint A — the first clean cut (concrete edits)

1) `scenebasic_uniform.h` — remove unused meshes/textures

Keep only what you still need:

```cpp
// Remove these:
std::unique_ptr<ObjMesh> R2Mesh;
std::unique_ptr<ObjMesh> C1Mesh;
std::unique_ptr<ObjMesh> XWingMesh;
```

```
// Remove these textures:
GLuint C1diffuseTexture, C1normalMap;
GLuint R2diffuseTexture, R2normalMap;
GLuint XWingDiffuseTexture, XWingNormalMap;
```

Keep:

```cpp
std::unique_ptr<ObjMesh> LightsaberMesh;
GLuint LSdiffuseTexture, LSnormalMap;

// (Optional) keep Plane/Cube if you want a stand/ground,
// otherwise remove those too.
Plane plane;
Cube cube;

// Skybox: remove for now to simplify
SkyBox sky;
GLuint cubeTex;
GLSLProgram prog, skyboxShader;
```

If you're going for "minimal first", I'd also remove **SkyBox** + `skyboxShader` + `cubeTex` in this checkpoint.

---

2) `SceneBasic_Uniform::SceneBasic_Uniform()` — only load the lightsaber mesh

```cpp
SceneBasic_Uniform::SceneBasic_Uniform()
    : tPrev(0), angle(0.0f), rotSpeed(glm::pi<float>() / 2.0f),
      plane(100.0f, 100.0f, 100, 100),
      teapot(14, glm::mat4(1.0f)),
      torus(1.75f * 0.75f, 0.75f * 0.75f, 50, 50),
      sky(100.0f)
{
    LightsaberMesh = ObjMesh::load("media/Lightsaber_01.obj", true);
}
```

Then delete the other `ObjMesh::load(...)` lines.

(If you remove torus/teapot/sky later, also delete them from the constructor init list.)

---

3) `initScene()` — only load the lightsaber textures

Delete these loads:

```cpp
C1diffuseTexture = ...
C1normalMap = ...
R2diffuseTexture = ...
R2normalMap = ...
XWingDiffuseTexture = ...
XWingNormalMap = ...
cubeTex = Texture::loadCubeMap(...)
```

Keep only:

```cpp
LSdiffuseTexture = Texture::loadTexture("media/texture/Lightsaber_01_lambert1_BaseColor.png");
LSnormalMap     = Texture::loadTexture("media/texture/Lightsaber_01_lambert1_Normal.png");
```

If you keep skybox for now, leave cubeTex; otherwise remove all skybox stuff in `compile()` + `render()` too.

---

4) `render()` — delete everything except the lightsaber draw

You want to end up with something roughly like this:

```cpp
void SceneBasic_Uniform::render()
{
    prog.use();
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Light setup (keep your directional light)
    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f, -0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));

    prog.setUniform("Light.L",  vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    // Bind lightsaber textures to match your shader bindings
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);
```

```
    // Material (tweak later)
    prog.setUniform("Material.Ks", vec3(0.5f));
    prog.setUniform("Material.Ka", vec3(0.8f));
    prog.setUniform("Material.Shininess", 80.0f);

    // Lightsaber transform
    model = glm::mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    // model = glm::scale(model, vec3(...)); // if needed

    setMatrices();
    LightsaberMesh->render();
}
```

Then delete:

- the cube render
- all XWing-follow-camera code
- C1 render
- torus render
- plane/skybox blocks (depending on whether you keep them)

This is the biggest "mess reduction" with the least risk.

One important shader clean-up (optional, but worth it)

In `basic_uniform.frag` you have:

```glsl
layout (binding = 2) uniform sampler2D DiffuseTex2;
```

...but you're not using it. Once you've cut down the C++ side, remove that uniform too (and any code that referenced it). It's not harmful, just noise.

Next: rotating the hilt (Checkpoint B)

Once Checkpoint A compiles and shows only the hilt:

Don't rotate the whole camera. Rotate the model matrix.

Add in the header:

```C++
float hiltYaw = 0.0f;
float hiltPitch = 0.0f;   // optional
bool rmbDownPrev = false;
```

Then in `updateCamera(dt)` (or better: rename later), instead of WASD flying:

- read mouse deltas while RMB is held
- adjust `hiltYaw/hiltPitch`
- keep view fixed

Then in `render()`:

```C++
model = glm::mat4(1.0f);
model = glm::rotate(model, glm::radians(hiltYaw), vec3(0,1,0));
model = glm::rotate(model, glm::radians(hiltPitch), vec3(1,0,0));
model = glm::rotate(model, glm::radians(270.0f), vec3(1,0,0));
```

That gives you a "product viewer" style setup.

Next: blade toggle (Checkpoint C)

Lowest-risk blade:

- render a long thin **Cube** as the blade
- use a **separate unlit shader** (so it's always bright)
- toggle with a key (edge-triggered like your ESC logic)

If you want, we can do it in two tiny files:

- `shader/blade.vert`
- `shader/blade.frag`

...and add `GLSLProgram bladeProg; bool bladeOn; bool bWasDown;`

Before we move on

You've shown the core files already, so next step is straightforward:

1. Do you want to **remove the skybox** in Checkpoint A, or keep it as background?
   Either is fine — removing it makes the cleanup faster.

If you tell me "remove it" or "keep it", I'll give you the exact edits for `compile()` + `render()` to match, and then we'll move straight into the hilt rotation + blade toggle.

okay hang on i havent removed anything yet, firstly i want to change the camera to be locked in place looking at the centrepoint of the scene, where the lightsaber hilt is being rendered, i know how to remove the models myself thats not an issue for me

Got you — let's lock the camera as a fixed "product viewer" camera that always looks at the hilt's centre, and stop your free-fly code from fighting it.

What we'll do (minimal, safe)

1. Set a **fixed camera position** and **fixed target** (the hilt centre).
2. **Stop calling** `updateCamera()` (or make it do nothing for now).
3. Build `view` from `lookAt(camPos, target, up)` every frame (or once if nothing moves).

---

1) Pick a centre point for the hilt

Right now you render the hilt at:

```cpp
model = mat4(1.0f);
model = glm::translate(model, vec3(-5.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
```

So a sensible "centrepoint" target is:

```cpp
glm::vec3 hiltTarget(-5.0f, 0.0f, 0.0f);
```

(We can refine later if the mesh pivot isn't centred.)

---

2) Lock the camera in `initScene()`

Replace your `view = glm::lookAt(...)` line with something like this:

```cpp
// Fixed camera looking at the hilt
glm::vec3 hiltTarget = glm::vec3(-5.0f, 0.0f, 0.0f);
camPos = glm::vec3(-5.0f, 2.5f, 7.5f);      // tweak these numbers to taste
camUp  = glm::vec3(0.0f, 1.0f, 0.0f);

view = glm::lookAt(camPos, hiltTarget, camUp);
```

Optional: since you're no longer doing FPS-style control, also disable cursor capture:

```cpp
cursorCaptured = false;
GLFWwindow* win = glfwGetCurrentContext();
if (win) glfwSetInputMode(win, GLFW_CURSOR, GLFW_CURSOR_NORMAL);
```

---

3) Stop the free-fly update from overwriting your view

In `update(float t)` you currently do:

```cpp
updateCamera(deltaT);
render();
```

For now, just comment out the camera update:

```cpp
// updateCamera(deltaT);
render();
```

That's the cleanest "locked camera" switch with the least risk.

---

4) (Optional but nice) Rebuild `view` each frame from the same locked setup

If later you rotate the hilt and want the camera to stay fixed: fine.
If later you add orbit/zoom, you'll rebuild view each frame.

For now, if you want it explicit, put this at the top of `render()` (or in `update()`):

```cpp
glm::vec3 hiltTarget = glm::vec3(-5.0f, 0.0f, 0.0f);
view = glm::lookAt(camPos, hiltTarget, camUp);
```

Not required if nothing changes, but it makes it harder for old code to "leak in".

Quick sanity checks

- If the hilt isn't centred in the camera view, adjust **camPos** first (Z distance, Y height).
- If it's clipped, increase the distance or adjust your `perspective` near plane (you currently use `0.3f`, which is fine).

If you paste the bit of your `render()` where you position the lightsaber (just that small section), I can suggest a better `camPos`/target pair that frames it nicely — but the steps above will lock it immediately without touching your shaders or model code.

okay so, i need a function that allows me to rotate the main model when we press w or s (up or down)
here is the model in question:
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, LSnormalMap);

//prog.setUniform("Material.Kd", 1.0f, 1.0f, 0.0f);
prog.setUniform("Material.Ks", vec3(0.5f));
prog.setUniform("Material.Ka", vec3(0.8f));
prog.setUniform("Material.Shininess", 80.0f);

model = mat4(1.0f);
model = glm::translate(model, vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(0.0f), vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));

setMatrices();
LightsaberMesh->render();

its this line i want to rotate with the up/down and w/s:
model = glm::rotate(model, glm::radians(0.0f), vec3(0.0f, 1.0f, 0.0f));
how best would i do this

Yep — the clean way is: **store the rotation angle as a member variable**, update it each frame based on key input, then use that value in your `glm::rotate(...)` line instead of `0.0f`.

You *don't* want to read key state inside `render()` if you can avoid it — keep input in `update()` (or a small helper), and keep `render()` purely drawing.

---

1) Add a hilt rotation variable (and speed) to your header

In `scenebasic_uniform.h`, add:

```cpp
float hiltYawDeg = 0.0f;          // rotation around Y axis (degrees)
float hiltYawSpeed = 90.0f;       // degrees per second (tweak)
```

(If you later want up/down tilt, that'd be a separate `hiltPitchDeg`.)

---

2) Update it from W/S each frame

In `SceneBasic_Uniform::update(float t)` you already compute `deltaT`. After that, add:

```cpp
GLFWwindow* win = glfwGetCurrentContext();
if (win)
{
    if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS)
        hiltYawDeg += hiltYawSpeed * deltaT;

    if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS)
        hiltYawDeg -= hiltYawSpeed * deltaT;

    // Optional: keep it bounded so it doesn't grow forever
    if (hiltYawDeg > 360.0f) hiltYawDeg -= 360.0f;
```

```
    if (hiltYawDeg < 0.0f)   hiltYawDeg += 360.0f;
}
```

That's it. This is "hold W to rotate one way, hold S the other way".

If you *literally* want **Up/Down arrows OR W/S**, just add extra checks:

```cpp
if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)   hiltYawDeg += hiltYawSpeed * deltaT;
if (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS) hiltYawDeg -= hiltYawSpeed * deltaT;
```

---

3) Use the variable in your model matrix

Replace your current line:

```cpp
model = glm::rotate(model, glm::radians(0.0f), vec3(0.0f, 1.0f, 0.0f));
```

with:

```cpp
model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f, 0.0f));
```

Everything else can stay the same.

Small but important note about rotation order

You currently do:

1. rotate X by 270
2. rotate Y by (your controllable value)
3. rotate Z by 300

That's totally fine, but remember **order matters**. If it ever feels like the hilt is rotating around a "weird axis", the fix is usually to change the order (or combine the fixed rotations into one "base orientation" and apply the user rotation either before or after it).

If you want the Y rotation to feel like it's spinning around the hilt's "natural" up-axis after you've oriented it, you'd typically do:

```cpp
model = mat4(1.0f);
model = glm::translate(model, vec3(0.0f, 0.0f, 0.0f));

// base orientation first
model = glm::rotate(model, glm::radians(270.0f), vec3(1,0,0));
model = glm::rotate(model, glm::radians(300.0f), vec3(0,0,1));

// then user spin last (often feels nicest)
model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0,1,0));
```

But don't change this unless it feels wrong — your current ordering might already match your mesh axis.

If you tell me whether you want **tap-to-step rotation** (each key press rotates e.g. 5 degrees) vs **hold-to-rotate**, I can give you the edge-triggered version too (like your ESC logic).

2/2

okay thats brill, although something seems weird as the lighting is staying attached to the model?as if its been baked in, since when i rotate the model, the *dark side* of the model stays and will rotate to the top of the model

Thought for 15s

Yeah — that "lighting rotates with the model" look almost always means **you're accidentally transforming the light direction by the model matrix**, so the light ends up in *model space* (effectively baked to the object).

In your case it's coming from this exact combo:

In `render()` you set a *directional* light in view space

```cpp
glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f, -0.4f));
glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));
```

✅ That's fine: `Light.Position.xyz` is now a direction in view space.

But in your vertex shader, you rotate it by `ModelViewMatrix` again

```glsl
if (Light.Position.w == 0.0)
    lightDir = normalize(mat3(ModelViewMatrix) * Light.Position.xyz); // <-- problem
```

Because `ModelViewMatrix = view * model`, that line applies the **model rotation** to the light direction — so when the hilt rotates, the light direction rotates with it.

Fix (minimal, correct for your current setup)

Change the directional-light branch in `basic_uniform.vert` to **not use ModelViewMatrix**:

```glsl
vec3 lightDir;
if (Light.Position.w == 0.0)
    lightDir = normalize(Light.Position.xyz);      // directional already in view space
else
    lightDir = Light.Position.xyz - Position;      // point light in view space

LightDir = invTBN * lightDir;
ViewDir  = invTBN * normalize(-Position);
```

That's it. After this, when you rotate the model, the highlight/shadow should stay "in the world/camera", not glued to the mesh.

Why this is the right fix (in your pipeline)

- `Position` is computed in **view space**:

  ```glsl
  vec3 Position = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;
  ```

- `TBN` (and therefore `invTBN`) is derived from `NormalMatrix` built from `mv`, so it's also effectively in **view space**.
- Therefore `LightDir` should be in **view space** *before* you convert it into tangent space with `invTBN`.

You already pass the light direction in view space, so multiplying by `mat3(ModelViewMatrix)` was the double-transform.

Quick extra note (only if you see odd shading with scaling)

Your C++ `NormalMatrix` isn't the true inverse-transpose — it's just the top-left 3×3 of MV:

```C++
prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]), vec3(mv[2])));
```

If you later scale non-uniformly, normals can go weird. The robust version is:

```C++
prog.setUniform("NormalMatrix", glm::mat3(glm::transpose(glm::inverse(mv))));
```

But if you're only rotating (and uniform scaling), you're fine.

If you make that shader change and it still feels "stuck", tell me whether you're rotating around Y *before or after* those fixed X/Z rotations — because axis order can make the "dark side" look like it's moving strangely even with correct lighting.

okay thats brill, lighting seems to be a bit washed out and bright now, the blinnphong doesnt look as it does in the labs, could you help troubleshoot this for me?

Thought for 13s ›

Yep — now that the light is behaving "correctly", the *wash-out* usually comes from one (or a mix) of these:

1. **Ambient is way too strong** (most common)
2. **Your diffuse term ignores** `Material.Kd` (so your material controls don't behave like the lab's)
3. **No gamma correction / sRGB mismatch** (textures look brighter/flatter than expected)

Let's hit them in that order (small, safe changes).

---

1) Your ambient is currently huge

In C++ you set:

```cpp
prog.setUniform("Light.La", vec3(0.25f));
prog.setUniform("Material.Ka", vec3(0.8f));
```

In your shader ambient is:

```glsl
vec3 ambient = Light.La * Material.Ka * texColour;
```

So ambient ≈ `0.25 * 0.8 = 0.20` of the texture **everywhere**, even in shadow. That's bright.

Quick fix (C++):

Try these values first:

```cpp
prog.setUniform("Light.La", vec3(0.08f));      // ambient light
prog.setUniform("Light.L",  vec3(0.9f));       // main light strength

prog.setUniform("Material.Ka", vec3(0.15f));   // ambient material
prog.setUniform("Material.Kd", vec3(1.0f));    // add this (see next section)
prog.setUniform("Material.Ks", vec3(0.25f));   // spec a bit lower
prog.setUniform("Material.Shininess", 80.0f);
```

(You're not currently setting `Material.Kd` at all — but you should.)

---

2) Your diffuse ignores `Material.Kd`

Right now you do:

```glsl
diffuse = texColour * sDotN;
return ambient + (diffuse + spec) * Light.L;
```

In the lab, it's usually closer to:

- diffuse is scaled by **Material.Kd**
- (optionally) specular is scaled by **light** and sometimes **texture** rules

Make your diffuse match the usual Blinn-Phong form

Change your `blinnPhong()` to:

```glsl
vec3 blinnPhong(vec3 n) {
    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Light.La * Material.Ka * texColour;

    vec3 s = normalize(LightDir);
    float sDotN = max(dot(s, n), 0.0);

    vec3 diffuse = Material.Kd * texColour * sDotN;

    vec3 spec = vec3(0.0);
    if (sDotN > 0.0) {
        vec3 v = normalize(ViewDir);
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }

    return ambient + (diffuse + spec) * Light.L;
}
```

And in your fragment shader uniforms you already have `Material.Kd`, so this is a clean correction.

Then make sure you actually set `Material.Kd` in C++ (see above).

---

3) If it's still "flat/bright": gamma / sRGB

If your lab output looked punchier, they may have been doing gamma correction (explicitly or via sRGB textures). Without it, things can look "off" and highlights feel wrong.

Two simple options:

Option A — quick shader gamma (easy test)

At the end of `main()` just before output:

```glsl
vec3 colour = blinnPhong(normalize(normal));
colour = pow(colour, vec3(1.0/2.2));   // gamma correct to display
FragColor = vec4(colour, 1.0);
```

If that suddenly looks more like the lab, you've confirmed gamma is a factor.

Option B — proper sRGB pipeline (later, slightly more involved)

- Load albedo/diffuse as sRGB internal format
- Enable `glEnable(GL_FRAMEBUFFER_SRGB);`

But try Option A first as a sanity check.

A super-fast troubleshooting checklist

Try these toggles one at a time:

1. **Kill ambient** temporarily:
   - set `Light.La = vec3(0.0f)`
     If the wash-out disappears instantly, ambient was the culprit.
2. **Kill spec** temporarily:
   - set `Material.Ks = vec3(0.0f)`
     If the wash-out is mainly from blown highlights, tune spec down / shininess up.
3. **Force a constant normal** (to rule out normal map issues):
   In frag: `vec3 normal = vec3(0,0,1);`
   If lighting suddenly looks right, your normal map or TBN handedness is making the surface face the light too much.

If you paste your current **C++ Light.L / Light.La / Material.Ka/Ks/Shininess** values after your latest changes, I can give you a tight set that matches the "lab look" (less ambient, clearer shading, controlled spec).